Study book

Topic 1: Setup + the C# bridge

ProjectHub learning track: React/TypeScript to C#/.NET backend. Week 1, topic 1 of 20.

Brief overview

What this lesson is about. You learned the core C# building blocks you need to read any backend code: how C# types differ from TypeScript, how null is handled, how classes and records differ, and how to model two entities (`Project`, `TaskItem`). You also ran the first experiments and used the debugger (breakpoint, step over, Variables panel) in Rider.

Why it matters for the interview. Almost every later topic (LINQ, EF Core, DTOs, tests) depends on these ideas. The questions 'class vs record', 'what does string? do', 'why decimal for money' are classic C# interview openers.

How to use this book. Read it top to bottom once, then use Part 9 (interview answers) and Part 10 (summary) to refresh. Everything here comes from our session: the lesson, the code, your questions, and the mistakes we corrected.

Contents

Part	What is inside
1	Environment and the project scaffold (what each file is, commands, cloud vs local)
2	C# types vs TypeScript types (nominal typing, numbers, value vs reference, enums, dates, decimal vs double)
3	Nullability (string?, int?, operators, CS8618/CS8602/CS8629, compile time vs runtime)
4	Classes: properties, required, init, initializers, collection expressions, naming rules
5	Records, equality, and what 'identity' means for entities and EF Core
6	Our code: TaskItemStatus, TaskItem, Project, explained line by line + design decisions
7	The experiments (playground) with code and expected output
8	Debugging in Rider: step-by-step instructions + what we learned
9	Questions you asked, mistakes we corrected, and interview Q&A with model answers
10	Summary, cheat sheet, and what comes next

Part 1. Environment and project scaffold

1.1 How we work: where things run

Thing	Where	Notes
Lessons, code writing, builds, quick experiments	Cloud session (claude.ai/code) or locally	The cloud container has .NET 10 installed by `scripts/cloud-setup.sh` and can run `dotnet build/run/test`.
Debugger (breakpoints, Watch, Call Stack)	Your Mac, in Rider	The cloud session has no visual debugger. This is why debugging labs are always local.
SQL Server	Docker: `docker compose up -d`	Needed from topic 8. In the cloud it is pulled in the background (large image). Ignore until topic 8.
Your editor	Rider (chosen in this session)	SSMS is Windows-only; for SQL use the VS Code mssql extension or DBeaver.

Good to know

Session types: a session started as a cloud session stays a cloud session; you cannot switch a local session to cloud (or back) in the middle. A new session is needed to change the type. Whether cloud usage is free or counts against your plan depends on your plan; check your claude.ai usage settings, nothing in the repo can tell.
Branches: the repo had two branches with the same starting commit. Your first commit (`unit 1`) landed on `claude/gallant-wozniak-mvix3u`; it was fast-forward merged into `claude/practical-bohr-1b5pb9`, which is now the one working branch. Pull it on your Mac and use it only.

1.2 What the scaffold contains

File	What it is	TS / Node analogy
`ProjectHub.slnx`	The solution: a list of projects	monorepo root / workspaces
`src/ProjectHub.Api/ProjectHub.Api.csproj`	Project file: target framework, NuGet packages, compiler options	`package.json` + `tsconfig.json`
`Program.cs`	App startup: registers services, builds the HTTP pipeline, `app.Run()`	`server.ts` with Express setup
`Controllers/WeatherForecastController.cs`	Template example endpoint (deleted in topic 6)	Express router
`appsettings*.json`	Configuration per environment	`.env` / config files
`Properties/launchSettings.json`	How `dotnet run` starts locally (ports, environment)	`scripts` in `package.json`
`Entities/` (new)	Our domain classes: `Project`, `TaskItem`, `TaskItemStatus`	`types/` or `models/` folder

1.3 Commands

```
dotnet build                              # compile; look for warnings, we want 0
dotnet run --project src/ProjectHub.Api   # start the API (port printed in console)
dotnet run playground.cs                  # .NET 10: run a single .cs file with no project
dotnet test                               # from week 3
docker compose up -d                      # SQL Server on localhost,1433 (from topic 8)
```

`<Nullable>enable</Nullable>` in the csproj is roughly `strictNullChecks`: it turns on nullable reference type analysis (Part 3). **Treat warnings as bugs.** The target for our files is 0 warnings.

Part 2. C# types vs TypeScript

2.1 Nominal vs structural typing (the biggest mindset change)

In TypeScript two types with the same shape are compatible: **structural** typing. In C# a value is compatible with a type only if it **is** that type or inherits from / implements it: **nominal** typing. The name of the type matters, not the shape.

C#

```
class A { public string Name { get; set; } = ""; }
class B { public string Name { get; set; } = ""; }

A a = new B();   // compile error: B is not A, same shape does not matter
```

TypeScript

```
// the same idea in TypeScript compiles fine:
class A { name = ""; }
class B { name = ""; }
const a: A = new B();   // OK in TS (structural)
```

How to read `A a = new B();`: it is `const a: A = new B()` in TS. Type first, then the variable name, then `= value`. You said this syntax was unfamiliar; this is the whole trick. Interview one-liner: 'C# uses nominal typing, TS structural.' Practical consequence: you will write explicit DTO classes/records and mapping code instead of passing 'something that looks right'.

2.2 Numbers: TS has one, C# has several

C# type	Use for	TS equivalent / note
`int`	ids, counts (32 bit)	`number`
`long`	very big ids/counts	`number` / `bigint`
`decimal`	**money**: exact decimal arithmetic	no built-in equivalent; literals need `m`: `19.99m`
`double`	measurements, math (binary floating point)	same as JS `number`
`bool`, `string`, `char`	as expected; `char` is one character `'a'`	
`DateTime` / `DateTimeOffset`	dates and times	`Date` (but not the same, see 2.5)
`Guid`	UUIDs	`crypto.randomUUID()` string

`var x = 5;` is **type inference**, like `const x = 5` in TS. `x` is still a strongly typed `int` forever. `var` here is NOT the old JS `var`.

2.3 Value types vs reference types

	Value types	Reference types
Examples	`int`, `bool`, `decimal`, `DateTime`, `Guid`, `enum`, `struct`	`class`, `record`, `string`, arrays, `List<T>`

	Value types	Reference types
Variable holds	the value itself	a reference (pointer) to an object, like JS objects
Assignment	copies the value	copies the reference (both point to the same object)
Can be null?	No (unless `int?`)	Yes at runtime (compiler warns if you did not allow it)
Default value	`0`, `false`, `default(DateTime)`	`null` (NOT empty string for `string`!)

`string` is a reference type but immutable, and `==` compares its **content**, so it feels like a JS string.

Common mistake
In the quiz you answered 'string is ""' when asked about the default value. Wrong: the default of `string` is `null`. The default of `int` is `0`. This is also why an uninitialized `int count;` shows `0` in the debugger.

No `undefined` in C#. Every field has a default value, and the compiler knows every property of a type, so there is no 'missing property'.

2.4 Enums

```
public enum TaskItemStatus { Todo, InProgress, Done }   // stored as int 0, 1, 2

var s = TaskItemStatus.Done;
```

Like a TS `enum`. `default(TaskItemStatus)` is `0`, i.e. `Todo`, so even without an explicit default a new task would be `Todo`. We still write `= TaskItemStatus.Todo` so the intent is explicit. There is no direct equivalent of `'todo' | 'done'` string unions; enums are what C# uses. Naming rule: `Todo` (PascalCase, one word). We renamed `ToDo` to `Todo` because this name will show up in JSON and the database.

2.5 Dates: DateTime, UtcNow vs Now, and a correction

Common mistake
You guessed 'DateTime.Now is an int, milliseconds since 1970'. That is **JavaScript** (`Date.now()`, Unix time). In .NET, `DateTime` is a struct holding **ticks** (units of 100 nanoseconds since 0001-01-01) plus a `Kind` flag: Utc, Local or Unspecified.

	`DateTime.Now`	`DateTime.UtcNow`
Meaning	current time in the server's local time zone	current time in UTC, independent of time zone
Problem / benefit	same moment reads differently on different servers; breaks with daylight-saving shifts and when the server moves regions	same moment everywhere; the standard for storing timestamps in a database
Rule	avoid on backends	use for `CreatedAt` and similar

Interview bonus: `DateTimeOffset` additionally stores the UTC offset and many teams prefer it for timestamps. Unset `DateTime` is `0001-01-01`, which is why `CreatedAt` needs a default (an initializer `= DateTime.UtcNow`).

2.6 decimal vs double (money)

`double` stores numbers in **binary**, exactly like a JS `number`. You know `0.1 + 0.2 === 0.30000000000000004` in JS: the same happens with `double` in C#. Tiny errors add up in prices and totals. `decimal` stores exact **decimal** numbers, so `0.1m + 0.2m == 0.3m`. It is slower, but money needs exactness.

Common mistake

In the quiz your reason was 'because it is the type for money'. That is the conclusion, not the reason. The reason: binary floating point cannot represent most decimal fractions exactly, decimal can.

Part 3. Nullability

With `<Nullable>enable</Nullable>` the compiler tracks whether a reference can be null.

```
string name = null;          // warning CS8600: you said it cannot be null
string? description = null;  // OK: '?' means "may be null"

int count = null;            // ERROR: value types are never null...
int? maybeCount = null;      // OK: ...unless you write int? (that is Nullable<int>)
```

Operators (same ideas as TS)

```
int len = description?.Length ?? 0;    // ?. and ??  work like in TS
description ??= "none";                // ??=  assign if null
if (description is null) { }           // preferred null check
if (description is not null) { }
string sure = description!;            // '!' = "trust me" (TS non-null assertion). Avoid.
```

What `string?` actually changes

	`string?`	`int?`
Kind of type	reference type: can already hold null, `?` is only a **compiler annotation**	value type: can never be null; `int?` is a **real different type**, `Nullable<int>`
At runtime	`string` and `string?` are the **same type**; nothing is enforced	has `.HasValue` and `.Value`; `.Value` on null throws `InvalidOperationException`
Effect	compile-time warnings only	changes the actual type

Remember

Answer to 'what does `string?` change: compile time, runtime, or both?' Compile time only. It is like TS types, which are also erased. At runtime `string` can still be null (from JSON or a database), and then you get `NullReferenceException`, the C# version of 'Cannot read properties of undefined'.
Why `string` can hold null but `int` cannot: `string` is a reference type (a reference can point to nothing), `int` is a value type (the value itself is stored; there is no 'nothing').

Warnings you will meet constantly

Code	Message	Meaning / fix
CS8618	Non-nullable property must contain a non-null value when exiting constructor	You declared `string Name` but nothing guarantees it gets set. Fixes: (1) `required`, (2) a default value (`= ""`, `= []`), (3) set it in a constructor, (4) make it `string?` but only if null is a real, valid state. Choose by meaning.
CS8602	Dereference of a possibly null reference	You used `x.Something` where `x` may be null (our experiment: `maybeName.Length`). A **warning**, not an error: the build succeeds and the crash happens at runtime.
CS8629	Nullable value type may be null	You used `x.Value` on an `int?` that may be null.

In our entities: `Title` is `required`, `Description` is `string?` (it is genuinely optional), `Tasks` gets `= []`. For `Tasks`, an empty list means 'no tasks', so null never needs to be a state, and callers can write `project.Tasks.Count` or `.Add(...)` without null checks.

Part 4. Classes

```
public class Project
{
    public int Id { get; set; }                                    // property with get/set
    public required string Name { get; set; }                      // must be set at creation
    public string? Description { get; set; }                       // optional
    public DateTime CreatedAt { get; init; } = DateTime.UtcNow;    // set once, then read-only
    public List<TaskItem> Tasks { get; set; } = [];                // never null
}

var p = new Project { Name = "Website" };   // object initializer, like { name: "Website" }
p.Name = "Website v2";                      // fine: it has set
// p.CreatedAt = DateTime.UtcNow;           // ERROR: init-only
// var bad = new Project { };               // ERROR: required member 'Name' not set
```

The pieces

- **Property** `{ get; set; }`: not a plain field, but a pair of accessor methods the compiler generates. Public data in C# is always properties. Fields (`private int _count;`) are private internals.
- `required`: the compiler **refuses to build** if the caller creates the object without setting this property. It is like a non-optional property in a TS interface. It is not about defaults.
- `init`: the property can be set **only while the object is being created** (in `new X { ... }` or a constructor) and is read-only afterwards. Like TS `readonly`, except you can set it in the initializer. It is also not about defaults; the default value comes from the separate `= DateTime.UtcNow`.
- **Initializer** `= value`: runs at creation. `= TaskItemStatus.Todo`, `= []`, `= DateTime.UtcNow`.
- **Collection expression** `[]` (C# 12+): creates an empty `List<T>` here. Same idea as `= []` in TS.
- **Object initializer** `new X { A = 1 }`: like `{ a: 1 }`.
- **Access modifiers**: `public`, `private` (default for members), `protected`, `internal` (visible inside the project).
- **File-scoped namespace** `namespace ProjectHub.Api.Entities;`: the modern style, one line, no braces.

Common mistake

In the first quiz you answered `required` = 'requires the field in a constructor' and `init` = 'gives a default value'. Corrected: `required` = must be set at creation or the build fails; `init` = settable only during creation, read-only after. After the correction you answered correctly: 'value can be set only on initialization, and after that it is blocked from modifying'.

Conventions

- PascalCase for types, properties and methods; camelCase for locals and parameters; `_camelCase` for private fields.
- One public type per file, file named after the type. This is why `TaskItemStatus` moved to its own file `TaskItemStatus.cs`.
- Why `TaskItem` and not `Task`? `System.Threading.Tasks.Task` is the type behind async/await (topic 4). Naming ours `Task` would collide and confuse every reader.

Part 5. Records, equality and identity

```
// positional record: one line
public record ProjectSummary(int Id, string Name, int TaskCount);

var a = new ProjectSummary(1, "Web", 3);
var b = new ProjectSummary(1, "Web", 3);
Console.WriteLine(a == b);                  // True: compared by VALUES

var c = a with { TaskCount = 4 };  // non-destructive copy, like { ...a, taskCount: 4 }
Console.WriteLine(c);  // ProjectSummary { Id = 1, Name = Web, TaskCount = 4 }
```

A one-line record generates: a constructor, `init`-only properties, **value equality**, a readable `ToString()`, and deconstruction. `with` creates a **new** object; the original is not changed (you confirmed this in the experiment).

	JS / TS	C#
Reference equality (same object in memory?)	`a === b` on objects	default for `class`; `ReferenceEquals(a, b)`
Value equality (same contents?)	`isEqual` / `JSON.stringify`	default for `record`

5.1 Class vs record: the interview answer

	`class`	`record`
Equality	reference: is it the same object?	value: do they hold the same data?
Mutability	usually mutable (`set`)	usually immutable (`init`), copy with `with`
Meaning	a thing with **identity** that changes over time	a **value** or a data snapshot
In a .NET backend	**entities** (EF Core tracks and changes them): `Project`, `TaskItem`	**DTOs** (request/response contracts), messages, results: e.g. `ProjectSummary` (appears in topic 6)

Rule of thumb: an entity with an `Id` that gets updated is a class. Data moving across a boundary (HTTP in/out) is a record. `struct` / `record struct` also exist (value types for small values); you will rarely write them in backend code.

5.2 What 'identity' means (your question)

You asked: 'what is an identity? objects of a class can be created and changed, yeah?' Yes, class objects can be created and changed. The question is how two objects are compared and how the system knows they are 'the same thing'. There are two kinds of identity:

- **Reference identity**: which object in memory. Classes have it by default.
- **Entity identity**: which database **row**. For an entity this is the primary key `Id`. Task 5 is still task 5 after you rename it or change its status. (Your earlier answer 'identified by unique reference' was close, but the entity's identity is its `Id`, not its memory address.)

```
var a = new PersonClass { Name = "Anna" };
var b = new PersonClass { Name = "Anna" };
a == b;    // class: False (two objects).   record: True (same values)
```

Why EF Core wants entities as classes

1. **Change tracking.** You load a task, change `Status`, call `SaveChanges()`. EF compares the tracked object with a snapshot and generates an `UPDATE`. This works because the object stays the same object while its properties change.
2. **One object per row.** Inside one `DbContext`, loading task 5 twice returns the **same instance** (the identity map). That relies on reference identity.
3. **Records break this.** Value equality makes two tasks with identical data look 'equal' although they are two different rows. And the hash code of a mutable record changes when you edit a property, which can corrupt `HashSet` / `Dictionary` lookups.

Remember

One sentence for the checkpoint: a **record is a value** (equal if the contents are equal); a **class entity is a thing with identity** (the same task even as its contents change).

Part 6. Our code, explained

These are the final files in `src/ProjectHub.Api/Entities/`. They build with **0 warnings, 0 errors**.

```
// TaskItemStatus.cs
namespace ProjectHub.Api.Entities;

public enum TaskItemStatus
{
    Todo,
    InProgress,
    Done
}
```

```
// TaskItem.cs
namespace ProjectHub.Api.Entities;

public class TaskItem
{
    public int Id { get; set; }
    public int ProjectId { get; set; }
    public required string Title { get; set; }
    public TaskItemStatus Status { get; set; } = TaskItemStatus.Todo;
    public DateTime CreatedAt { get; init; } = DateTime.UtcNow;
}
```

```
// Project.cs
namespace ProjectHub.Api.Entities;

public class Project
{
    public int Id { get; set; }
    public required string Name { get; set; }
    public string? Description { get; set; }
    public DateTime CreatedAt { get; init; } = DateTime.UtcNow;
    public List<TaskItem> Tasks { get; set; } = [];
}
```

6.1 Design decisions: why each line looks like this

Line	Why
`class`, not `record`	Entities have identity (`Id`) and change over time; EF Core tracks them (Part 5).
`public int Id { get; set; }`	Primary key. EF Core finds a property named `Id` (or `ProjectId` on `Project`) by convention. `set` because EF assigns it when it loads/inserts a row.
`int ProjectId` on `TaskItem`	Foreign key: which project this task belongs to. The scalar FK is how the database sees the relation.
`List<TaskItem> Tasks` on `Project`	The other side of the relation (navigation property). We will use it in topic 11 (`Include`).
`required string Title`	A task without a title makes no sense: the compiler forces every creator to provide it, and CS8618 is satisfied.
`string? Description`	Genuinely optional, so `null` is a valid state and is stated in the type.
`Status = TaskItemStatus.Todo`	New tasks start as Todo. Explicit, even though `default` would also be 0.

Line	Why
`CreatedAt { get; init; } = DateTime.UtcNow`	A creation timestamp must not change, so `init`. UTC so it is time zone independent.
`Tasks = []`	Never null, so callers never need a null check.

6.2 Review of the first version you pushed (commit 'unit 1')

The first version built with 0 warnings but had issues an interviewer or a code reviewer would notice. All were fixed:

Finding	Fix
`Project.cs` was the lesson example pasted with its comments	Cleaned up. Lesson: you must be able to explain every property, not only paste it.
Enum value `ToDo` instead of `Todo`	Renamed to `Todo` (spec + .NET naming; appears later in JSON and DB).
Two public types in `TaskItem.cs`	Enum moved to `TaskItemStatus.cs` (one public type per file).
`CreatedAt` was `init` in `Project`, plain `set` in `TaskItem`	Both `init`: a creation timestamp should not change.
Trailing spaces, double space in `= TaskItemStatus`, no final newline	Reformatted (Rider: Code > Reformat Code).
(minor) `Tasks { get; set; }` lets a caller replace the whole list	Left as is for now; a `get;`-only property is a design option we can discuss later.

Part 7. The experiments (playground)

Goal: see the difference between class and record, `with`, and `int?` with your own eyes. It is a **single file** run with no project: 'scratch file' simply means a throwaway file that is not part of the repo.

How to run it (Mac)

1. Open a terminal and run `mkdir ~/playground && cd ~/playground` (outside the repo, nothing gets committed).
2. Create `playground.cs` in that folder and paste the code below.
3. Run `dotnet run playground.cs` (works with .NET 10).

```
// ===== Step 1: class vs record equality =====
var c1 = new PersonClass { Name = "Anna" };
var c2 = new PersonClass { Name = "Anna" };
Console.WriteLine($"1a. class == class:   {c1 == c2}");

var r1 = new PersonRecord("Anna");
var r2 = new PersonRecord("Anna");
Console.WriteLine($"1b. record == record: {r1 == r2}");

// ===== Step 2: 'with' copies a record =====
var r3 = r1 with { Name = "Bob" };
Console.WriteLine($"2.  original: {r1}   copy: {r3}");

// ===== Step 4: nullable int =====
int? x = null;
Console.WriteLine($"4a. x.HasValue: {x.HasValue}");
try
{
    Console.WriteLine(x.Value);
}
catch (Exception ex)
{
    Console.WriteLine($"4b. runtime error: {ex.GetType().Name}: {ex.Message}");
}

// ===== Step 3: nullable string (just look at the build warnings) =====
string? maybeName = null;
Console.WriteLine(maybeName.Length);   // the compiler warns here

class PersonClass { public required string Name { get; set; } }
record PersonRecord(string Name);
```

Expected output (verified in the cloud container)

```
warning CS8629: Nullable value type may be null.            (the x.Value line)
warning CS8602: Dereference of a possibly null reference.   (maybeName.Length)
1a. class == class:   False
1b. record == record: True
2.  original: PersonRecord { Name = Anna }   copy: PersonRecord { Name = Bob }
4a. x.HasValue: False
4b. runtime error: InvalidOperationException: Nullable object must have a value.
Unhandled exception. System.NullReferenceException:
    Object reference not set to an instance of an object.
```

What each result teaches

Line	Lesson
1a False	Two class instances with identical data are two different objects: reference equality.
1b True	Records compare by value.
2	`with` makes a copy; `r1` is untouched (immutability, like `{ ...r1, name: "Bob" }`).
4a / 4b	`int?` is `Nullable<int>`; `.Value` on null throws `InvalidOperationException`.
CS8602 + crash	Nullable warnings are compile-time hints. Ignore them and the same line crashes at runtime with `NullReferenceException` (JS: 'Cannot read properties of undefined'). Remove the `?` on `maybeName` to see the warning change.

Your answers after running it: class `==` is false because the objects are different references in memory, records compare by values; `r1` was not mutated; CS8602 says 'Dereference of a possibly null reference' on `maybeName.Length` because `maybeName` is null. All correct. Note: it is a warning, not an error: the build succeeds.

Part 8. Debugging in Rider (step by step)

Goal: stop the program at a chosen line, look at the values of variables, step line by line. This is how you find a bug without `Console.WriteLine`. It is done **locally** on your Mac (the cloud session has no visual debugger).

8.1 The exercise

1. **Add temporary code.** Open `src/ProjectHub.Api/Program.cs`. Right before the last line `app.Run();` paste the 4 lines below.
2. **Set the breakpoint.** Click in the grey gutter left of the line numbers next to `var count = ...`. A red dot appears.
3. **Start the debugger.** At the top right of Rider choose the `ProjectHub.Api` run configuration (http or https) and click the **bug icon** (Debug) next to the green play button. The app starts and stops at the red line, which is highlighted.
4. **Inspect.** The Debug panel opens at the bottom. In the **Threads & Variables** tab expand `project`, then `Tasks`: `Count = 2`, two `TaskItem`s with `Title`, `Status` (`Todo`) and `CreatedAt`. Hover over `project` in the code to see it as a tooltip.
5. **Step Over** once and watch `count` change.
6. **Resume** to continue the program.
7. **Clean up.** Delete the 4 temporary lines and the breakpoint. Nothing from this is committed.

```
var project = new ProjectHub.Api.Entities.Project { Name = "Website" };
project.Tasks.Add(new ProjectHub.Api.Entities.TaskItem { ProjectId = 1, Title = "Design" });
project.Tasks.Add(new ProjectHub.Api.Entities.TaskItem { ProjectId = 1, Title = "Build" });
var count = project.Tasks.Count;   // <- breakpoint goes on this line
app.Run();
```

8.2 Debugger controls (Rider on macOS)

Control	What it does	Where
Debug (bug icon)	start the app with the debugger attached	top right, next to the green play button
Step Over	run the current line, stop at the next line in the same method	Debug panel > Threads & Variables tab toolbar (curved arrow over a dot), or menu Run > Debugging > Step Over (shows your real shortcut)
Resume (green play)	continue until the next breakpoint or the end	vertical toolbar on the left of the Debug panel
Restart	stop and start the whole debug session again	same vertical toolbar; not needed in this exercise
Variables	values of locals, expandable objects	Threads & Variables tab

Shortcut note

The keyboard shortcuts I first gave you (F8, F9, Ctrl+D) did not match your Mac setup: F-keys usually act as media keys on macOS unless you hold **fn**, and Rider's macOS keymap differs. Use the toolbar buttons or look up the shortcut in the menu **Run > Debugging**.

8.3 What the screenshot taught: why `count` was 0

You asked why `count` showed `0` while stopped on `var count = project.Tasks.Count;` (the editor even showed `count: 0` inline). The answer:

- A breakpoint stops **before** the highlighted line runs. The yellow arrow means 'about to execute', not 'done'.
- At that moment `count` has not been assigned yet. A local `int` that is declared but not assigned shows its default value `0` (value types have non-null defaults).
- After **Step Over** the line executes and `count` becomes `2`.

8.4 What you observed (checkpoint answers)

Question	Your answer (correct)
What was `Status` of each task, and who set it?	`Todo`, set by the class's property initializer at the moment of creation, not by the caller.
What was `CreatedAt` and in which time zone?	Set at creation by `= DateTime.UtcNow`: UTC.
Why was `count` 0 before Step Over?	The breakpoint is before the line; the line was not executed yet; an unassigned `int` is 0.

Debugging vocabulary for later topics: **breakpoint** (where to stop), **Step Over / Into / Out** (line, enter method, leave method), **Watch** (an expression you pin to see), **Call Stack** (the chain of method calls that led here), **Locals/Variables** (current values). Topic 5 (Debug Lab A) practices these on a real bug.

Part 9. Your questions, corrections, and interview Q&A

9.1 Questions you asked during the session

Q: I cannot run 'Learning .NET web development' in cloud mode. Can you help, or should we continue here?

Continue here. In the cloud container `dotnet build/run/test` all work (.NET 10 is installed by the startup script). Only the visual debugger needs your Mac. The real constraint was that a session started as local cannot be switched to cloud (and the reverse); a new session is needed to change type.

Q: Why is the `count` variable zero at the breakpoint?

See 8.3: the breakpoint stops before the line runs, and an unassigned `int` is 0.

Q: What is an identity? Objects of a class can be created and changed, yes?

Yes. Identity is how the system knows two objects are 'the same thing'. Reference identity = same object in memory (class default). Entity identity = same database row, i.e. the same primary key `Id`. EF Core's change tracking and identity map depend on this (5.2).

Q: I did not understand `A a = new B();`. What is that syntax?

It is `const a: A = new B()` in TS. It fails in C# because typing is nominal: `B` is not `A`, even with identical properties. In TS it would compile (structural).

Q: What does `new file` / 'scratch file' mean? Give me normal instructions.

A throwaway file outside the repo. Create a folder, put `playground.cs` in it, run `dotnet run playground.cs` (Part 7).

Q: Should Claude write the code and I review it?

Decision made in this session: yes. Because your company uses AI for coding, the goal is to understand concepts and patterns and be able to explain and defend the code. `CLAUDE.md` now says 'reviewer mode'. Exception: the final ticket (topic 20) you do alone with docs only, as the independence check.

Q: Do we run this session in the cloud for free tokens?

Claude cannot see your plan or billing. The session runs in a cloud container; whether it is free or counts toward your limits depends on your plan. Check your claude.ai usage settings.

9.2 Mistakes we corrected (so you do not repeat them)

You said	Correct
`DateTime.Now` is milliseconds since 1970	That is JS `Date.now()`. .NET `DateTime` is ticks since year 0001 plus a Kind. `Now` = local time, `UtcNow` = UTC.
Default of `string` is `""`	Default is `null`. Default of `int` is `0`.
`required` = required in a constructor	`required` = must be set when creating the object, or the build fails.
`init` = gives a default value	`init` = settable only during creation, read-only afterwards.
`decimal` because it is the type for money	Because it is exact decimal arithmetic; `double` is binary floating point and has rounding errors.
Entity identity = 'unique reference'	Entity identity = primary key `Id`; reference identity is just which object in memory.
(copy-paste of lesson code as 'my' code)	Be able to explain every line; retype or review with understanding.

9.3 Interview questions with model answers

Q: Class vs record: differences and when to use each in a backend?

A class compares by reference and is usually mutable; a record compares by value, is usually immutable and copied with `with`. Use classes for entities (identity, change over time, EF Core tracking): `Project`, `TaskItem`. Use records for DTOs, messages and results: data crossing a boundary, e.g. `ProjectSummary`.

Q: Value type vs reference type, with examples?

Value types (`int`, `bool`, `decimal`, `DateTime`, `Guid`, enums, structs) hold the value and are copied on assignment; they cannot be null unless `T?`. Reference types (`class`, `record`, `string`, arrays, `List<T>`) hold a reference to an object; assignment copies the reference; they can be null.

Q: What do nullable reference types (`string?`) do at compile time vs runtime?

Compile time only: the compiler tracks null-state and warns (CS8600, CS8602, CS8618). At runtime `string` and `string?` are the same type; null can still appear (JSON, DB) and cause `NullReferenceException`.

Q: `string?` vs `int?`?

`string?` is an annotation on a reference type that can already be null. `int?` is `Nullable<int>`, a real different value type with `.HasValue` and `.Value`.

Q: What is CS8618 and how do you fix it?

A non-nullable property is not initialized when the constructor finishes. Fix with `required`, a default value / initializer, a constructor assignment, or make it nullable if null is a real state.

Q: `required` vs `init` vs a constructor parameter?

`required`: caller must set it at creation (compile error otherwise). `init`: can be set only during creation, read-only after. A constructor parameter forces the value through the constructor. They combine: `required string Title { get; init; }` means must be provided and cannot change.

Q: Structural (TS) vs nominal (C#) typing?

TS compares shapes, C# compares declared types: a `B` is not an `A` unless it inherits or implements it, even with identical members.

Q: `decimal` vs `double` for money?

`decimal`: exact base-10 arithmetic. `double` is binary floating point and cannot represent many decimal fractions exactly (0.1 + 0.2 != 0.3), errors accumulate.

Q: What does `with` do on a record?

Creates a new record copying all values, with the listed properties changed. The original is not modified.

Q: What is a property and how is it different from a field?

A property is a pair of get/set accessor methods generated by the compiler (public API, can have logic, `init`, `required`). A field is a raw private variable. Public data is exposed through properties.

Q: Why UTC for `CreatedAt`? Why not `DateTime.Now`?

UTC is time zone independent, so the same moment is the same everywhere and across daylight-saving changes; local time depends on the server. Consider `DateTimeOffset` when you need the offset.

Q: Why is an EF Core entity a class and not a record?

Entities have identity (primary key) and are mutated; EF tracks changes to the same object and keeps one instance per row per context. Value equality and a changing hash code of a mutable record break that model.

Q: Why `TaskItem` and not `Task`?

`Task` is the .NET type for async operations (`System.Threading.Tasks.Task`); reusing the name would collide and confuse.

Q: What is the difference between a breakpoint line being highlighted and the line having run?

A breakpoint stops before the line executes. The highlighted line is the next to run. Step Over executes it.

Part 10. Summary

Key takeaways

C# is **nominally typed** (TS is structural). Numbers: `int`, `long`, `decimal` (money), `double`. `var` is inference, not JS `var`.
Value types (`int`, `bool`, `DateTime`, `enum`, ...) hold values and are never null unless `T?`; **reference types** (`class`, `record`, `string`, `List<T>`) hold references and default to `null`.
Nullable reference types are compile-time only. Treat warnings (CS8618, CS8602) as bugs. Fix CS8618 with `required`, a default, a constructor, or `?`, chosen by meaning.
Properties `{ get; set; }`, `required` (must be set at creation), `init` (set only at creation), initializers (`= DateTime.UtcNow`, `= []`).
Class = reference equality, mutable, identity: entities. **Record** = value equality, immutable, `with`: DTOs.
An entity's identity is its **primary key**. EF Core tracks changes on class instances and keeps one object per row per context.
Store timestamps in **UTC**. Use `decimal` for money. Default of `string` is `null`, of `int` is `0`.
Debugger: breakpoint stops **before** the line; Step Over runs the line; Variables shows the state; Resume continues.
Entities built: `TaskItemStatus` (Todo/InProgress/Done), `TaskItem`, `Project`: 0 warnings.

Cheat sheet: TypeScript to C#

TypeScript	C#
`const x: T = ...`	`T x = ...;` or `var x = ...;`
`string | null`	`string?`
`x?.y ?? z`	`x?.y ?? z`
`readonly` prop	`{ get; init; }`
non-optional interface prop	`required`
`{ ...a, name: 'x' }`	`a with { Name = "x" }`
`a === b` (objects)	reference equality (`class`)
deep equal	`record` `==`
`Date.now()`	`DateTime.UtcNow` (different representation!)
`enum`	`enum`
`Array` / `[]`	`List<T>` / `[]`

Checklist: can you do this without notes?

- Explain class vs record and give one backend example of each.
- Explain what `string?` changes (compile time) and what stays the same (runtime).
- Explain CS8618 and name the four fixes.
- Explain `required` vs `init` with a code example of what the compiler rejects for each.
- Explain nominal vs structural typing and read `A a = new B();`.
- Explain `decimal` vs `double`, and `UtcNow` vs `Now`.
- Explain what 'identity' means for an EF Core entity.
- Use the debugger: breakpoint, start, Step Over, read Variables, resume.

What comes next: Topic 2 (Collections + LINQ)

`List<T>` is like an array, `Dictionary<K,V>` like a `Map`, and LINQ (`Where`, `Select`, `OrderBy`, `GroupBy`, `FirstOrDefault`) maps almost 1:1 to `filter`, `map`, `sort`, `reduce`, `find`. You will also meet `IEnumerable<T>` and deferred execution. Build: filter, sort, group and project sample ProjectHub data. Checkpoint: translate map/filter/find into LINQ.

Working agreement: every topic ends with a study book like this one (a thorough PDF with an overview at the start, everything we covered including your questions, and a summary at the end).